M.A.P - A Math Advancement Platform: Automated Generation of Adaptable Math Problems using Context-Free Grammars

Michael Seavers
School of Engineering and Applied Sciences
Western Kentucky University
Bowling Green, United States of America
Michael.Seavers622@topper.wku.edu

***Abstract*—Many students rely on textbooks to practice and improve their math skills; however, these textbooks have limited examples and problems, and many do not have solutions or clear steps to solve them. Online resources are better solutions as they provide step-by-step solutions; however, many often rely on question banks, which are costly for the company and limit the possibilities a problem can have. We propose M.A.P. - A Math Advancement Platform, which generates mathematical problem sets using greedy context-free grammar generation with difficulty adjustments. This approach provides the potential for unlimited problems and solves issues with traditional problem-generation solutions.**

***Index Terms*—adaptive learning, automated problem generation, dynamic difficulty adjustment, context-free grammar, grammar-based generation, greedy algorithm, and math education.**

I. Introduction

Many students find themselves having a difficult time when working on math problems. As time is limited in the classroom, they must practice independently, using textbooks or online resources; however, there is a distinct problem: limited possibilities. Textbooks can only have so many pages, resulting in a fixed number of example problems. In many cases, these problems are reused from other sources or do not give the full example, such as solutions or steps for solving. This can limit the students' learning possibilities as they must take time to find out where and why they went wrong.

Online resources are slightly better as many always provide step-by-step solutions; however, they are also limited by the number of problems. Many famous websites such as Khan Academy and IXL rely on both large question banks along with question generation [1]. These sites also apply other strategies to create problems. A standard solution is to use generative artificial intelligence (AI) or a large language model (LLM) [2]. These are excellent choices; however, training properly takes time and computational resources. In addition, these systems are not perfect and can generate flaws. An example is Oracle's GPT, where the answers provided are incorrect or not fully correct. Due to this, they either need to make further processing or review questions before sending them out to students.

IXL uses another common strategy, which relies on creating templates or rules to generate problems [3]. Although we do not know the exact templates or completed strategies, the typical approaches result in limited generations. Some limitations include predictability, repetitiveness, and superficial generation. A standard template for linear equations is $ax + b = c$, where a, b, and c are random values. Not only are the generations limited, but it does not provide the full context this topic can include, such as multiplicity and division.

Another significant disadvantage is static generation, where problems only follow the formula given. This generation type can work for certain problems, such as Pythagorean theorem calculations, but not in more complex expressions, such as limits and integrals. In these higher-level expressions, many types of rules become impossible to replicate or at least very repetitious. Websites that explore this type of generation use other algorithms and machine learning techniques to select the correct template or adapt the difficulty of the question in general.

Of course, we do not know the exact formulas, systems, and algorithms deployed by the websites as they are not open knowledge; however, both types of systems require enormous computational and labor costs. Question banks require large databases and time to find and select questions. Machine learning algorithms also take time to train but rely on large datasets to learn correctly. Templates require less cost, but they also perform the poorest independently. Other sources, such as selective algorithms, are required to increase the performance of these models.

Aside from the current web pages, researchers have also explored various new techniques for generating math problems. One such path is the use of context-free grammar (CFG). Traditionally, CFG is used in compilers to verify programming syntaxes and is solely used to tell if a particular expression is within a set of rules. Researchers have been exploring generating expressions using CFG,

This project was created for CS500. I thank Dr. Zhonghang Xia for his guidance throughout the CS500 project.

which is the opposite of what they are meant for. Some have used a probability approach [4], while others explored rule-iteration context [5]. We have also explored CFG and presented our approach for CFG generation using a greedy selection.

The rest of this paper is as follows: Section 2 contains background information on this project, including motivations, features, and design decisions. Section 3 reveals all the technical information, such as the question generation and difficulty evaluations. Section 4 will discuss the project in general, including its limitations, benefits, and problems discovered during implementation. Section 5 will conclude the paper by summarizing the project and giving future work directives.

II. Background

In this section, we introduce the motivations and directives involved in our project and system and share a general outline of all the features and decisions made.

A. Motivation

Many systems on the market help students practice their mathematics; however, many either lock many of the necessary features under a paywall, have a limited number of questions, or the learning mechanics are poorly designed for progression. Textbooks, too, have problems regarding the number of problems and their solutions being available in full detail, as mentioned in the introduction. Other sources, such as generative models such as Oracle's GPT, also have limited uses and provide a mix of both accurate and inaccurate information. Due to all of these, we wanted to implement a simple system that could easily generate problems with the ability to dynamically get more or less difficult as the user progresses. With a simple system designed, students could easily log in and start practicing question topics, such as integrals, and help them get more unique practices.

For the generation system, we choose context-free grammar almost out of a whim. We were introduced to the CFG concept in a automa class where we had the idea that if we can check if a language is in a grammar, we also produce a language from the grammar. We did very little research at the time and only discovered that most companies use other techniques, but after further research, we found papers using CFG to generate problem sets. That said, we could not find any papers using these models in depth or a greedy solution.

B. Goals

The are three major goals for the project.

1) Create a system that can generate mathematical questions with the ability to adjust their difficulty.
2) Create a user system that allows users to create accounts and log in.
3) Scale question difficulty based on user progression.

We also had some sub-goals that were meant for extras and user flexibility.

1) On-demand question addons/additions using an Admin user type.
2) Both user-friendly and professional web pages and designs.
3) Solutions generated in steps. (failed)

C. Features

Our software development model was a free-style agile methodology. At the start of the semester, we created a list of features and labeled mini-deadlines for each using a ghant chart. Below is a list of all the features that we have implemented.

- Webpages
  - Homepage (with login)
  - Dashboard
  - Topic/Subtopic Selector
  - Question Solver
  - Admin Panel
- Database Tables
  - Users
  - Rules
  - UserLogs
  - Topics
  - Question Types (subtopics)
  - Questions
- Systems
  - User Login / Signup
  - Topic/Subtopic adds/removes (admin)
  - Rules creation and deletions (admin)
  - Greedy CFG question generator
  - Progressive questions
- Security
  - Encrypted user data in the database
  - Hashed password system
  - Locked pages requiring the user to log in before access

We will discuss the features that could not be created in section 4.

III. Technical Details

In this section, we share all the technical details related to the project, including programming languages, resources used, database structure, how questions are generated, and adaptive difficulties.

A. Used Tools and Software

The front end of our system was developed using typescript and react. Instead of CSS, we used a popular library called Tailwind for animations and design templates. For displaying math expressions, we used the react version of latex, which displays them in LaTeX format. We used the React version of Lucid for some of our page symbols. The

rest of the code was created by hand or with the help of generative AI (labeled).

The backend of our system was developed using Python. We used fastAPI to connect our webpage and server. For API calls, we created pydantic models to easily accept requests. For database connections, we used sqlalchemy. We used bcrypt, and we used the cryptography library for hashing. Some scripts required multiple async functions, so we used asyncio for them. For regular expressions, we used re. Lastly, we used sympy to solve the math problems.

For our database, we used PostgreSQL. We did not use cloud services but installed the data onto our MacBook. Although some changes happened through the console, we tried strictly using pyalchemy for table generation and fixes.

B. Database Schema

We created a database to hold our users' information and save the question rulesets that admins create. There are six tables in total. We will only describe the three most important ones. The first table is the users table. The following are its attributes.

- user_id - integer (PK)
- user_username - string
- user_email - string
- user_password - string
- user_phone - string
- user_type - string

We did not add strict rules as we attempted to handle everything in Python. Thus, the backend holds many rules to prevent unwanted data from being stored. Again, all values except for id and type were hashed or encrypted.

Another table we created was for transaction logs. The following are its attributes.

- id - integer (PK)
- user_id - integer (FK)
- topic_id - integer (FK)
- question_type_id - integer (FK)
- timestamp - timestamp
- difficulty - float
- time_taken - float
- attempts - integer
- is_correct - boolean
- skipped - boolean

We also added relationships to the FKs for easier access to data during database querying.

The last table we created was a bridge table that made up our questions. Again, we did not store questions in a bank but autogenerate them. We use this table to store the rules so admins can dynamically update them. The following are its attributes.

- user_id - integer (PK)
- topic_id - integer (PK)
- question_type_id - integer (PK)

Again, we used relationships on all of the attributes for easier access to the data during querying.

The other tables created were rules, topic, and question_type table.

C. Question Generation

As discussed throughout this paper, we used context-free grammar to generate math problems. The grammar [6] is defined as $G = (\sum, V, T, S, P)$ where:

1) $\sum$ is a finite set of characters
2) V is a finite set of variables
3) T is a finite set of terminals
4) V and T are a subset of $\sum$
5) S is the starting variable
6) P is a set of rules, where a rule is a part of (State -> expression...)

For this project, we strictly use only the rules. Below is a simplified example of a rule set for linear equations. Assume S is the starting state.

```
Rule1: S -> E=E
Rule2: E -> E + T | E - T | T
Rule3: T -> id | x | id*x
```

In traditional CFG, we would use this rule to verify that this grammar accepts an input set; however, we do not need validation. We are looking for expression creation, but it is impossible to do normally. As shown in figure 1, rulesets can be depicted as a tree. The tree starts from

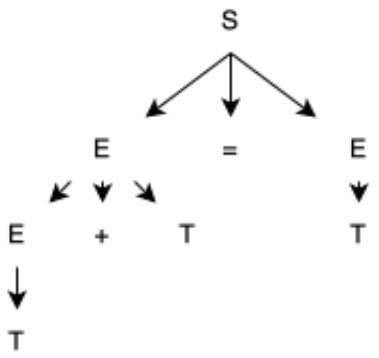

Fig. 1. Tree representation of linear equation ruleset

the starting node and expands from the left (or right) side. We can use any tree-searching algorithm to expand the expression, but we typically use depth-first search (DFS). The problem with just using this tree is that the tree can expand indefinitely. Various solutions address this problem, but we choose to use costs. We limit the number of possible expansions by assigning costs to each rule. Below is an example of the costs for the linear equation ruleset.

```
Rule1: S -> E=E (0)
Rule2: E -> E + T (1) | E - T (1) | T (0)
Rule3: T -> id (0) | x (1) | id*x (2)
```

The costs are shown as the number in the parentheses. Each rule's expression receives a cost value. Rules with only one ruleset can only have a cost of zero; otherwise, the generation would never end, or we would see a variable in the output. Also, to keep expansion balanced, we increase costs as they are used. A cost of two would turn to four, then six, and so on.

Once we have the costs, we can easily generate expressions. Our system takes in a maximum cost, and then, using a greedy algorithm, we can continue expanding the rules until we reach an output with only non-terminals. The greedy algorithm finds the path with the greatest cost for the variable. If the total cost has room for it, expand using that rule and increase the total cost. If not, find the next largest cost. This repeats until we find a cost that is zero.

This solution is not perfect. If you generate with it, the expressions are not always evenly balanced. Since the expression follows DFS, the expansion happens from left to right. This means the left-most variable must hit a terminal to stop the expansion. We solve this issue using two methods. The first method is to switch the target variable each time. In the first iteration, we look at the left-most variable. In the next iteration, we look at the right-most variable. We keep flipping back and forth until there are no more non-terminals. The second method uses a new value, priority. We assign each variable with a priority value. The algorithm now searches for the highest cost and highest priority.

For the example above, we gave Rule 3 a higher priority than the other rules. In this case, whenever a T is present, the algorithm will select the left or right-most T, ignoring the cost requirement (since T is 0). Another strategy we can use is to remove left recursion from our ruleset. Below is the updated rule without left recursion and priority.

```
Rule1 (0): S -> E=E (0)
Rule2 (0): E -> TE‘ (0)
Rule3 (0): E‘ -> +TE‘ (1) | -TE‘ (1) | empty (0)
Rule4 (1): T -> id (0) | x (1) | id*x (2)
```

In the example above, we now show the priority of each rule and have removed the left recursion. By applying all these methods, we can easily generate sets of problems. The next problem would be creating rulesets for each topic and subtopic pair. We must make them in a way that creates more difficult problems. For our generation, we defined a harder problem as one that uses higher-cost rules and a larger problem set (total rules).

D. Adaptive Difficulties

After creating the generator, we can now focus on determining when a user should have a more complicated, easier, or no change in their difficulty. Unfortunately, we do not have any data as this is a new system. We could create mock data, but we avoided creating data because our system doesn't require training. We gather new data and use that to help determine an individual user's difficulty ranking. We put the user's old data in our log table in our database. We keep track of the number of attempts it took the user to solve the problem, how long it took, if they skipped it, the current difficulty, and if they got it right or wrong. Our system only counts the questions wrong if they exceed three attempts.

For new users, we gave them a choice between easy, medium, and hard. These values are predetermined to be one, ten, and twenty. After answering the question correctly or incorrectly, the system determines whether they should increase, decrease, or stay the same. We first need to get the user's past data for a given topic and subtopic pair to determine this. We then look at only the logs with the same difficulty as the latest entry. We follow these steps.

1) Calculate average time taken T excluding the latest entry
2) Calculate average cost A of all logs excluding the latest entry
3) Calculate cost C of the latest entry
4) If $C > A$ + threshold, increase the difficulty
5) If $C < A$ - threshold, decrease the difficulty
6) Else, keep the difficulty the same.

Costs are generated by using the following factors:

- Is_correct -> +10 * factor if true | 0 if false
- Skipped -> +5 if false | 0 if true
- Attempts -> -1.5 * attempts (where attempts are 0, 1, or 2)
- Factor -> max($f2$, min($f3$, 1 - ((time_taken - average_time) * $f1$)))
- $f1$, $f2$, $f3$ -> weighted values as scaling factor, min factor, and max factor (0.02, 0.5, 2.0)

By using these steps and calculations, we can determine whether the user is steadily progressing on the current difficulty. If so, we increase the difficulty, but if not, we lower it. Due to not having a trained model or any past data, we require the user to complete five problems before calculating the difficulty. The following section will discuss why we selected this algorithm for our dynamic difficulty selection.

IV. Discussion

In this section, we share our thoughts on various aspects of the project, including system limitations, delays, missed features, and all the failures throughout the semester and the solutions used to fix them.

A. Failed Features

At the start of this project, we marked various features we wanted to implement. Section 2 discussed all of the implemented features. Below is a list of the features that were not able to be implemented.

- Difficulty adjustment through machine learning models
- Responsive email/phone account recovering system
- User settings

- Steps of solution
- Large number of topics/subtopics
- Working Math Input Like symbolab
- Security

One of the significant features of this project is the dynamic difficulty adjustment of the question generation. The plan was to incorporate machine learning models to handle the adjustments and to determine whether to increase or decrease the difficulty based on the user's performance. Unfortunately, there was not enough time to thoroughly test and train the model. The two models we tested were deep knowledge tracing (DKT) and Bayesian Knowledge Tracing(BKT). Although the BKT model wasn't strictly a machine learning algorithm, we would have to use a custom type that requires training for our project. Nonetheless, we opted for a non-machine learning solution due to the time constraint.

A dynamic email and phone recovery system was one of the earliest features we started implementing. Users would receive an email or phone text with a code that would allow them to reset their password. This was planned for use on the account recovering page and potentially on the user settings page. We stopped halfway into the implementation as the feature wasn't that important or related to the project's primary goals. We wanted to return after the other features were implemented but ran out of time.

After implementing the user accounts, we wanted to allow users to customize themselves more. We implemented a profile picture display, allowed users to upload images, and allowed them to change or delete their accounts if they desired. Time was another reason we didn't implement this system, but we kept the display as it looked nice. However, it only showed what the default profile image would look like.

Another feature we wanted to implement is a step system to show the various steps to solve a problem. Due to the time constriant, we were unable to start working on the feature, and mainly focused on question generation.

We implemented a few topics, subtopics, and rules; however, we planned to incorporate a lot more. We wanted this project to be more of a finished product; however, it became more of a showcase of our generation system. It took us half the semester to create and finalize the generation system. This does not include the machine learning attempts, which we took a lot of time trying to get to work. Due to the delays, we decided only to incorporate some topics and subtopics and focused on overall user design improvements. Additonally, we aimed to incorporate many types of topics; however, our system could only handle single variable solutions.

Lastly, we wanted to dive further into the system's security. Although we do not have any user data, the project focuses on theoretical user data. We wanted to have a plan or some implementation to protect users' data and ensure correct authorization throughout the pages. Some security features were implemented, but we wanted to add much more, like secure API connections.

B. Difficulties

Three significant difficulties were faced throughout this project related to the main goals. The first difficulty involves problem generation. We knew we wanted to incorporate CFG to generate problems, which was easy, but although we could generate expressions, they were mostly static. In other words, our system was no better than the traditional templates. We attempted various strategies, some involving weights while others making it dependent on machine learning, but the method we stuck with was using a greedy algorithm, as described in section 3.

The second difficulty we faced was incorporating machine learning for adaptive difficulty selection. Our question generation requires a max cost threshold, which determines the difficulty. Again, higher means a more complicated question. We planned to use tracing models, as discussed previously, to analyze students' past performance and determine if they are ready to move on or need easier questions. We also planned to let the model adjust weights, favoring certain parts of the rules over others. For instance, the model would learn that a student did better at multiplication problems and would produce more division so the student could get more practice with them. As discussed in the previous subsection, we ran out of time attempting to incorporate the models we found. We did program them into the system but ran into training issues. After training our models, we found they were not 'learning' and memorizing instead. This could be due to the lack of real user data or programming errors. As the deadline approached, we switched to a non-machine learning solution and focused the project on the generation.

The third was creating the CFG rules. Our project only works with linear equations due to time constraints. We knew the solution however. We needed to add mini-codes into the expressions, such as $var and other unique commands that could process other infomation. We lack the time due to the normal generation.

C. Comparision and Contribution

In the beginning, as previously discussed, we did little research and only found the type of solutions major websites, such as IXL, use to generate or select questions. Again, these companies mostly use a large dataset of questions and special algorithms to select the most appropriate question for a user. Some sites also use machine learning models, such as generative AIs, to create new questions. Other sides use templates and more algorithms to ensure the templates are generating ideal problem sets.

These companies' solutions are ideal as they have been running for a long time, have a large amount of user data, and have perfected their solutions for their needs. Our system is designed to create problem sets of similar

levels but with fewer resources. You require an enormous amount of data to create question banks or train models. Our system uses no data to generate questions; however, for difficulty determination, we do require data. That said, the amount of data required is significantly less as we only need a portion of the data these companies have collected, which is only used for difficulty adjustments.

Near the end of the semester, we explored other papers within this field and found many others using CFG to generate problem sets. Although the idea is not original, to our knowledge, we are the only ones performing CFG using a greedy approach (costs). Unfortunately, we do not have enough time to compare the differences deeply to see which system is better. In the future, we wish to continue with this analysis.

As for contribution, all code can be found on our GitHub repository [7]. Although we used various libraries, as described above, most of the code was written from scratch. We also used Oracle's GPT4 and Google's Gemini in some code and design generation. All code generated has been commented as such.

V. Conclusion

To conclude, we have created a new system, M.A.P., for math problem generation. Our system uses both context-free grammar and greedy algorithms to generate questions. We then explore a non-machine learning approach for dynamically adjusting the difficulty. Although our system is far from perfect, there is some potential for later development.

A. Future Works

We see some potential in our system and plan to continue working on the project. We hope to explore various machine learning algorithms and apply them to our difficulty calculator. By doing so, we can have a better captive system that learns from the users, but again, we would need to collect actual data. We also want to explore another system, mainly large language models. We believe another solution is creating a massive model trained on thousands of mathematical textbooks. Through this research, we hope to create a better way for students to learn math.

Acknowledgment

This project was created for the CS500 (Research Methods) at Western Kentucky University under the direction of Dr. Zhonghang Xia. We also used some generative artificial intelligence models throughout the project. Any such cases were labeled in the code comments.

References

[1] "Free Math Worksheets," *Khan Academy Blog*, [Online]. Available: https://blog.khanacademy.org/free-math-worksheets/. [Accessed: April 29, 2025].

[2] Khan Academy, "AI Generators for Teachers," *Khan Academy Blog*, [Online]. Available: https://blog.khanacademy.org/ai-generators-for-teachers/. [Accessed: April 29, 2025].

[3] IXL, "How many questions are there? - IXL," *IXL Help Center*, [Online]. Available: https://www.ixl.com/help-center/article/1274270/how_many_questions_are_there. [Accessed: April 29, 2025].

[4] U. Primožič, L. Todorovski, and M. Petković, "Probability of deriving an algebraic expression with a probabilistic context-free grammar," arXiv preprint, arXiv:2212.00751v2, [Online]. Available: https://arxiv.org/abs/2212.00751v2. [Accessed: April 29, 2025].

[5] M. M. R. A. Milani, S. Hosseinpour, and H. Pehlivan, "Rule-Based Production of Mathematical Expressions," *Mathematics*, vol. 6, no. 11, Art. no. 254, Nov. 2018, doi: 10.3390/math6110254.

[6] J. E. Hopcroft and J. D. Ullman, *Introduction to Automata Theory, Languages, and Computation*. Boston, MA, USA: Pearson, 2007.

[7] Michael Seavers, "CS500-Project." Github, 2025, https://github.com/Mseavers1/CS500-Project.